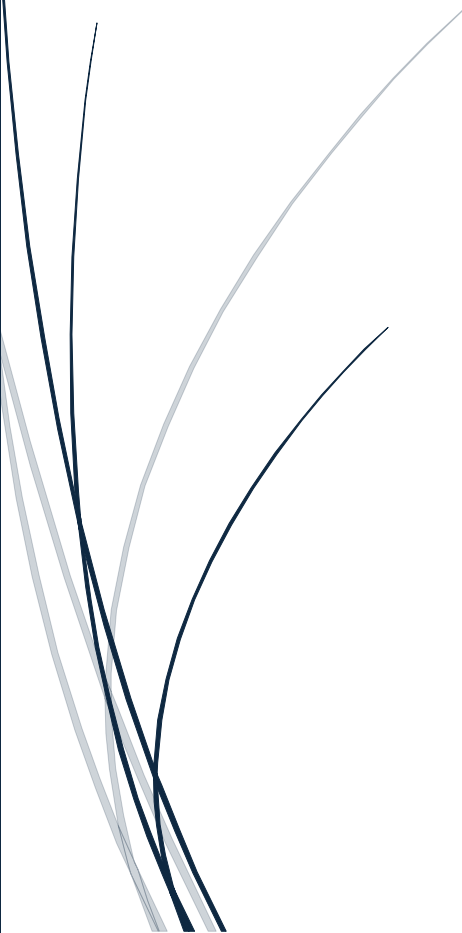




2-7-2026

# Solution Architect BP

Diseño de Arquitectura -  
Sistema de Banca por Internet



Benny Torres  
DEVSU

## CONTENIDO

### 1. Introducción y Contexto del Proyecto

#### a. Contexto del Negocio

La entidad bancaria BP opera una plataforma digital de servicios financieros que busca modernizar su arquitectura para soportar un crecimiento significativo en volumen de usuarios y transacciones. El sistema actual —asumido como un sistema monolítico o legacy existente— presenta limitaciones en escalabilidad, performance y mantenibilidad, especialmente durante picos de demanda (ej. campañas de adquisición masiva o cierres de mes).

El desafío principal para BP es rediseñar y construir una plataforma de banca por internet que contemple tres capacidades centrales: (1) consulta histórica de movimientos y transacciones - **Gestión de Cuentas**, (2) transferencias entre cuentas propias e interbancarias - **Motor de Transacciones**, y (3) **OnBoarding digital** con reconocimiento facial y apertura de cuenta. Estas capacidades deben operar de forma integrada, segura y escalable, bajo los estándares regulatorios del sector financiero peruano.

#### b. Objetivos del Proyecto

- Diseñar e implementar una plataforma de banca por internet que cubra los flujos de consulta de movimientos, transferencias propias e interbancarias, y onboarding digital biométrico.
- Mejorar significativamente la escalabilidad y performance del sistema frente al baseline actual.
- Implementar una arquitectura moderna, mantenible y orientada al dominio (DDD).
- Garantizar altos estándares de seguridad y cumplimiento normativo (SBS, PCI-DSS, Ley 29733).
- Reducir la complejidad operativa y el time-to-market de nuevas funcionalidades.
- Preparar la plataforma para crecimiento futuro: multi-país, nuevos productos financieros.
- Migrar progresivamente desde el sistema legacy sin afectar la continuidad operativa.

#### c. Alcance

El alcance principal incluye el diseño e implementación de los siguientes flujos: OnBoarding digital con reconocimiento facial y validación KYC, autenticación y gestión de identidad, consulta histórica de movimientos, transferencias entre cuentas propias e interbancarias, notificaciones y auditoría transaccional completa.

Se asume que el banco opera actualmente un sistema monolítico o legacy; la transición hacia la nueva arquitectura se realizará progresivamente mediante el patrón Strangler Fig, garantizando zero downtime y continuidad de servicio durante la migración.

Se considera fuera de alcance para esta fase: la migración completa de sistemas legacy no relacionados con los flujos mencionados, y la implementación de productos de crédito o inversión.

## 2. Análisis de Requerimientos

### a. Requerimientos Funcionales

- Onboarding digital completo con reconocimiento facial biométrico y liveness detection.
- Consulta histórica de movimientos y transacciones financieras (propias e interbancarias).
- Transferencias entre cuentas propias e interbancarias con validación en tiempo real.
- Validación de identidad (DNI, selfie, liveness detection) integrada a proveedores externos (Reniec, Buró de Crédito, SBS).
- Apertura de cuenta digital al finalizar el onboarding.
- Gestión de usuarios, perfiles y métodos de autenticación (clave, huella, facial).
- Integración con pasarelas de pago y canales de notificación (email, SMS, push).
- Auditoría completa e inmutable de todas las operaciones del cliente.
- Soporte para flujos de recuperación de cuenta y revisión por backoffice (aprobación/rechazo de casos KYC).

### b. Requerimientos No Funcionales

- **Funcionalidad:** Soporte multi-dispositivo (Web SPA + App Móvil), multi-idioma.
- **Usabilidad:** Experiencia de usuario fluida, con un tiempo de onboarding completo menor a 3 minutos.
- **Confiabilidad:** Disponibilidad  $\geq 99.95\%$ , manejo de fallos parciales, resiliencia e integridad de datos garantizada.
- **Performance:** Tiempo de respuesta del flujo de onboarding  $\leq 8$  segundos en el percentil 95. Soporte de más de 5,000 usuarios concurrentes en picos de demanda.
- **Escalabilidad:** Escalabilidad horizontal automática (HPA y Cluster Autoscaler sobre AKS).
- **Seguridad:** Cumplimiento de normativas financieras (SBS, PCI-DSS si aplica). Protección de datos personales (Ley 29733). Autenticación fuerte con MFA y modelo Zero Trust. Cifrado en reposo (AES-256) y en tránsito (TLS 1.3). Cumplimiento de soberanía de datos.
- **Mantenibilidad y Observabilidad:** Alta observabilidad con trazabilidad distribuida (correlation ID end-to-end), facilidad de mantenimiento y despliegue independiente por servicio.

### c. Requerimientos Normativos y de Cumplimiento

- Cumplimiento de regulaciones financieras peruanas (SBS): gestión de riesgo operacional, continuidad de negocio y reportes de auditoría.
- Ley de Protección de Datos Personales (Ley 29733): minimización de datos en logs y tokens, consentimiento explícito del titular.
- Buenas prácticas de ciberseguridad: OWASP Top 10, modelo Zero Trust, segmentación de red con Private Endpoints.
- Auditoría y trazabilidad completa e inmutable para todas las transacciones y acciones del cliente (no repudio).
- Estándares de identidad digital: OAuth 2.0 (RFC 6749), PKCE (RFC 7636), OpenID Connect.

### 3. Arquitectura de Alto Nivel

#### a. Visión general de la solución propuesta

La solución propone una plataforma de banca por internet estructurada en torno a **tres dominios de negocio principales**. La descomposición en microservicios se fundamenta en el estándar **BIAN (Banking Industry Architecture Network) v12**, que define Service Domains con responsabilidades, límites e interfaces explícitos para el sector financiero. Este enfoque evita una descomposición arbitraria y garantiza que cada microservicio tenga un propósito de negocio claro, alineado a prácticas reconocidas de la industria bancaria.

Los tres dominios principales son:

##### **Dominio de Identidad y Onboarding**

Agrupar los servicios de registro, validación biométrica, KYC y apertura de cuenta. Es el dominio de mayor complejidad en integraciones externas. Se mapea a los Service Domains: *Customer Onboarding*, *Customer Agreement*, *Know Your Customer (KYC)* y *Party Authentication*.

##### **Dominio de Operaciones Bancarias**

Agrupar los servicios de consulta de movimientos, ejecución de transferencias propias e interbancarias, y orquestación de pagos. Es el dominio de mayor criticidad transaccional. Se mapea a los siguientes Service Domains:

- **Savings Account** (BQ: Transaction, Balance): producto principal de apertura en el onboarding digital retail peruano — cuentas de ahorros con reglas de interés, capitalización y restricciones de retiro propias.
- **Current Account** (BQ: Transaction, Balance): complementario, para clientes con cuentas corrientes existentes — demanda de sobregiro, chequeras y uso empresarial.
- **Payment Initiation, Payment Order, Payment Execution**: cadena de orquestación para transferencias propias entre cuentas del mismo titular.
- **Payment Initiation, Payment Order, Correspondent Bank**: misma cadena extendida con el SD de corresponsalía bancaria para transferencias interbancarias hacia otras entidades financieras.

**Savings Account** y **Current Account** se implementan como microservicios independientes, dado que cada uno encapsula reglas de negocio propias e incompatibles entre sí: cálculo de intereses, capitalización periódica y restricciones de retiro en el caso de ahorros; sobregiro autorizado, emisión de chequeras y cargos por mantenimiento en el caso de cuenta corriente. Fusionarlos en un único servicio violaría el principio de Bounded Context de DDD y diluiría los límites explícitos que establece BIAN entre ambos Service Domains. Esta decisión se detalla en el capítulo 4 (Decisiones Arquitectónicas).

##### **Dominio de Soporte Transversal**

Agrupar los servicios de notificaciones, auditoría y gestión de sesión. Sirve a los dos dominios anteriores de forma desacoplada mediante mensajería asíncrona. Se mapea a los Service Domains: *Customer Notification* y *Financial Transaction History*.

El mapeo completo de Service Domains a componentes de implementación se detalla en el Diagrama C4 de Nivel 3 (sección 5).

Cada microservicio se estructura internamente bajo **arquitectura hexagonal (Ports & Adapters)** y principios de **Domain-Driven Design (DDD)**, aislando la lógica de dominio de los adaptadores de infraestructura (bases de datos, mensajería, servicios externos), lo que permite cambiar proveedores sin impactar el núcleo de negocio.

La plataforma se despliega sobre **Microsoft Azure (AKS)**, aprovechando servicios gestionados — Azure API Management, Azure Cache for Redis, Azure Key Vault, Azure Monitor + Application Insights — para reducir la carga operativa y garantizar los atributos de calidad requeridos.

Para la mensajería y el backbone de eventos se adopta una estrategia de dos capas con roles bien diferenciados:

- **Confluent Platform (Apache Kafka)** actúa como backbone principal de Event-Driven Architecture. Su modelo de log distribuido, inmutable y replayable lo hace idóneo para eventos de dominio entre microservicios, auditoría transaccional (alineada al SD Financial Transaction History de BIAN, que es naturalmente append-only) y escenarios de alto throughput (>5,000 usuarios concurrentes). Confluent Schema Registry garantiza además que los contratos de eventos entre Service Domains evolucionen de forma backward-compatible, aspecto crítico en una arquitectura bancaria donde múltiples consumidores dependen del mismo stream.
- **Azure Service Bus** se mantiene en un rol complementario y acotado: comandos que requieren dead-letter queue, reintentos con backoff garantizado y patrones de compensación transaccional (Saga), donde la semántica de entrega garantizada de cola es más apropiada que el modelo de streaming.

La transición desde el sistema legacy se realiza mediante el patrón **Strangler Fig**: las nuevas capacidades se exponen progresivamente a través del API Gateway, mientras el sistema legacy queda detrás de un Facade que se va retirando por fases, garantizando zero downtime en todo momento.

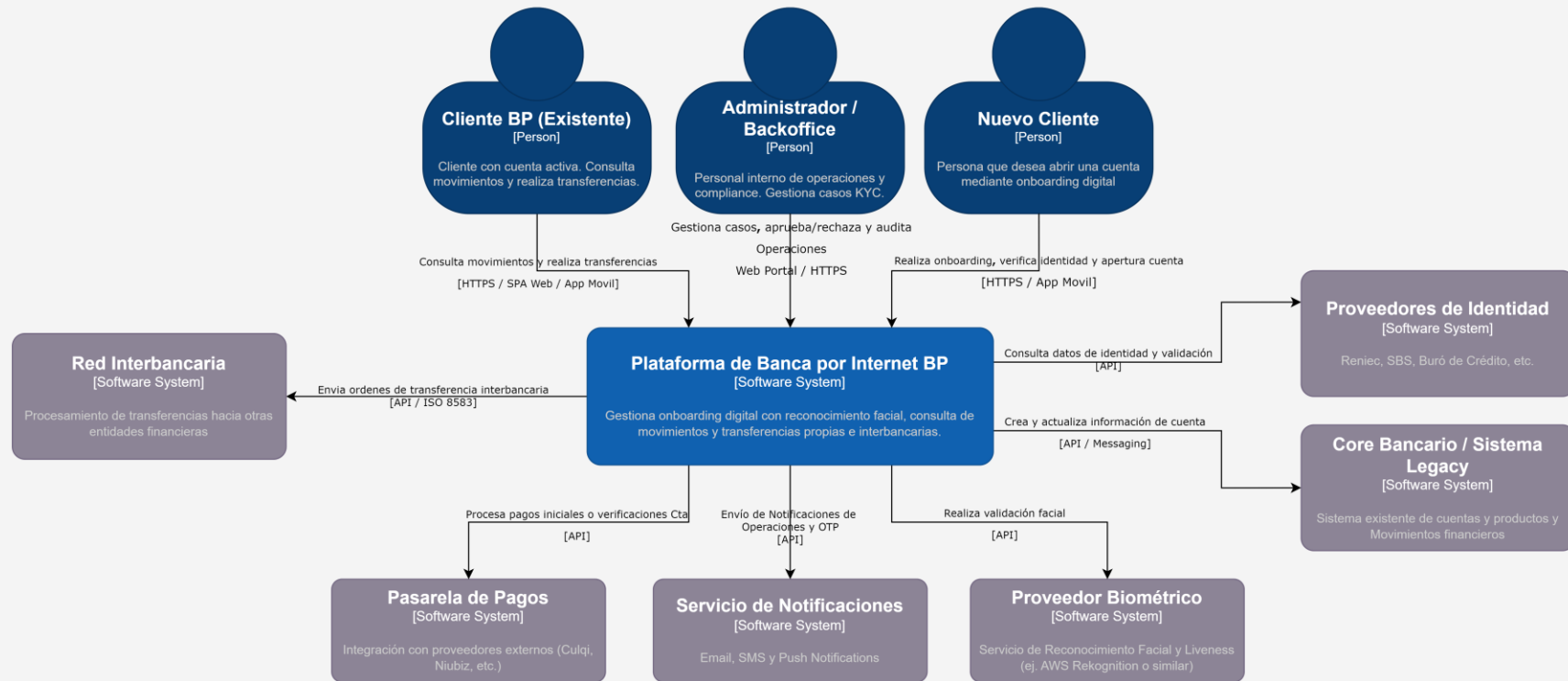
#### **Principios arquitectónicos clave:**

- Descomposición basada en BIAN Service Domains (v12) para límites de dominio bien definidos.
- Desacoplamiento fuerte mediante DDD Event-Driven Architecture sobre Confluent Platform (Apache Kafka) como backbone principal de streaming de eventos.
- Escalabilidad horizontal automática con AKS (HPA + Cluster Autoscaler).
- Seguridad por diseño: Zero Trust, OAuth 2.0 + PKCE, mTLS entre servicios internos.
- Observabilidad completa: Azure Monitor + Application Insights + trazas distribuidas con correlation ID end-to-end.
- Migración progresiva con Strangler Fig para garantizar continuidad operativa.

## b. Diagrama C4 – Nivel 1 (Contexto)

### Diagrama de Contexto - Plataforma de Banca por Internet BP

Nivel 1 C4: Actores, sistema principal y sistemas externos con los que interactúa



4. Decisiones Arquitectónicas Críticas

Las siguientes decisiones arquitectónicas constituyen el marco de referencia técnico para la plataforma de banca digital de BP. Estas han sido seleccionadas priorizando la escalabilidad, la seguridad transaccional y el cumplimiento normativo bancario. La estrategia adoptada busca equilibrar el cumplimiento de los requerimientos funcionales con estándares de la industria, evaluando activamente las alternativas para mitigar riesgos técnicos y operativos.

| Decisión Arquitectónica                          | Justificación Principal                                                             | Alternativas Consideradas                    | Decisión Final                                                                             |
|--------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------|
| Estilo Arquitectónico                            | Alineado con BIAN v12, límites de dominio claros y escalabilidad independiente.     | Monolito (rápido pero no escalable).         | Microservicios + Hexagonal + DDD.                                                          |
| Consistencia                                     | Integridad en flujos distribuidos (transferencias).                                 | 2PC (Transacciones distribuidas).            | Patrón SAGA (Orquestado).                                                                  |
| Seguridad Transaccional                          | Previene duplicidad ante reintentos de red/sistema.                                 | Reintentos simples sin control.              | Idempotencia (Idempotency Keys).                                                           |
| Frontend Web                                     | Excelente UX, ecosistema maduro y despliegues independientes.                       | Angular vs. Vue.                             | React + TypeScript.                                                                        |
| Frontend Móvil                                   | Código compartido con web y buen soporte para SDKs bancarios.                       | Flutter vs. Nativo.                          | React Native.                                                                              |
| Autenticación                                    | Seguridad en clientes públicos y estándar de la industria.                          | JWT simple vs. Session-based.                | OAuth 2.0 + OIDC + PKCE.                                                                   |
| API Gateway                                      | Centralización de políticas y seguridad bancaria.                                   | Kong vs. AWS API Gateway.                    | Azure API Management.                                                                      |
| Persistencia                                     | Escalabilidad independiente, Independencia de datos y optimización de lecturas.     | Base de datos compartida.                    | Database per Service + CQRS.                                                               |
| Caché                                            | Alto rendimiento en consultas frecuentes                                            | Cosmos DB Cache.                             | Azure Cache for Redis.                                                                     |
| Integración y Mensajería                         | Alta concurrencia y trazabilidad de eventos.                                        | Solo Service Bus.                            | Kafka (backbone) + Service Bus (Saga)                                                      |
| Cloud Provider                                   | Integración nativa y experiencia del equipo.                                        | AWS vs. Multicloud.                          | Microsoft Azure.                                                                           |
| Auditoría                                        | No repudio y cumplimiento regulatorio.                                              | Logs síncronos a BD.                         | Inmutable (Append-only).                                                                   |
| Backend for Frontend (BFF)                       | Optimización de respuesta por dispositivo y reducción de complejidad en el cliente. | Orquestación directa desde el API Gateway.   | BFF (Microservicios de Orquestación)                                                       |
| Alta Disponibilidad y Resiliencia                | Garantiza continuidad (SLA 99.95%) y protección ante fallos zonales/regionales.     | Infraestructura única zona vs. Nube híbrida. | Multi-zona (AKS) + GRS (RPO/RTO).                                                          |
| Resiliencia, Excelencia Operativa y Auto-healing | Mitiga fallos en cascada y garantiza disponibilidad mediante auto-recuperación.     | Reintentos simples vs. Monitoreo pasivo.     | Circuit Breaker (Resilience4j) + Auto-healing (K8s) + Azure Monitor + App Insights (SLOs). |

## a. Análisis Detallado de Decisiones Arquitectónicas

En esta sección se fundamentan las decisiones técnicas tomadas para la plataforma de banca digital de BP, asegurando la alineación con los estándares de la industria (BIAN), la seguridad y la resiliencia operativa.

### i. Estilo Arquitectónico: Microservicios + DDD + Hexagonal

- **Justificación 1:** El dominio bancario posee subdominios con ciclos de escalabilidad muy distintos (ej. consulta de movimientos es de alto volumen/baja complejidad, mientras que transferencias es baja frecuencia/alta criticidad). La microsegmentación permite escalar y desplegar estos servicios de forma independiente, optimizando costos operativos.
- **Justificación 2:** La arquitectura hexagonal aísla la lógica de negocio (**reglas de dominio BIAN**) de los adaptadores de infraestructura. Esto es crítico para permitir el cambio de proveedores de biometría o bases de datos con un impacto mínimo en el núcleo del negocio.

### ii. Consistencia Transaccional: Patrón SAGA (Orquestado)

- **Justificación 1:** Garantiza la integridad de datos en procesos de larga duración (transferencias interbancarias) mediante una secuencia de transacciones locales y compensaciones ante fallos, evitando el uso de bloqueos distribuidos (2PC).
- **Justificación 2:** El enfoque orquestado centraliza la trazabilidad del estado de la transacción, facilitando la auditoría regulatoria (SBS) al permitir observar el progreso y compensación de cualquier transferencia compleja.

### iii. Seguridad Transaccional: Idempotencia (Idempotency Keys)

- **Justificación 1:** Previene errores de duplicidad en transacciones críticas ante reintentos de red del cliente o del sistema, asegurando que un mismo pago no sea procesado dos veces.
- **Justificación 2:** Facilita la resiliencia en arquitecturas basadas en eventos, donde un consumidor podría recibir el mismo evento múltiples veces; la validación de claves asegura que el estado final del sistema sea siempre correcto.

### iv. Frontend Web: SPA (React + TypeScript)

- **Justificación 1:** Separa completamente el ciclo de vida del frontend del backend, permitiendo despliegues independientes y reduciendo el blast radius de cambios accidentales en la experiencia de usuario.
- **Justificación 2:** Facilita la reutilización de componentes de UI y lógica compartida con el equipo móvil, acelerando el time-to-market al estandarizar el desarrollo bajo un mismo stack tecnológico.

### v. Frontend Móvil: React Native

- **Justificación 1:** Permite compartir lógica de negocio con la SPA web, además de contar con un soporte maduro para SDKs bancarios (biometría, OAuth, Push) en la región, reduciendo riesgos de integración.
- **Justificación 2:** El mercado de talento regional es más amplio para JavaScript/TypeScript en comparación con Dart (Flutter), reduciendo el riesgo de bus factor y facilitando la escalabilidad del equipo de desarrollo.



vi. Autenticación: OAuth 2.0 + OIDC + PKCE

- **Justificación 1:** Tanto SPAs como apps móviles son *public clients*; el uso de PKCE (Proof Key for Code Exchange) mitiga ataques de interceptación del código de autorización, garantizando un flujo seguro frente a cualquier redirect URI comprometido.
- **Justificación 2:** Es el estándar actual recomendado por la RFC 8252, ofreciendo el nivel de seguridad necesario para datos financieros y cumpliendo con las mejores prácticas de protección contra interceptación de tokens.

vii. API Gateway: Azure API Management

- **Justificación 1:** Proporciona un punto único de entrada para la implementación de políticas transversales como throttling, autenticación centralizada y observabilidad, simplificando la gestión de las APIs expuestas.
- **Justificación 2:** Su integración nativa con Azure permite utilizar Private Endpoints, alineándose con la estrategia Zero Trust de BP al mantener el tráfico interno aislado de la red pública.

viii. Persistencia: Database per Service + CQRS

- **Justificación 1:** Evita el acoplamiento de datos entre microservicios, lo que permite que cada equipo evolucione su esquema de base de datos sin afectar a otros dominios, garantizando independencia total.
- **Justificación 2:** El uso de CQRS (Command Query Responsibility Segregation) permite separar las operaciones de escritura (comandos) de las de lectura (consultas), optimizando el rendimiento para consultas históricas sin sobrecargar el servicio transaccional.

ix. Caché: Azure Cache for Redis (Cache-Aside)

- **Justificación 1:** El patrón Cache-Aside minimiza el riesgo de inconsistencia en datos que cambian frecuentemente (como saldos y movimientos), operando con una latencia de milisegundos para lecturas de alta frecuencia.
- **Justificación 2:** Proporciona resiliencia para las sesiones de usuario y descarga de trabajo al Core bancario (sistema legacy), protegiéndolo de picos de consultas de lectura.

x. Integración y Eventos (Confluent Platform / Kafka):

- **Justificación 1:** Confluent Platform (Kafka) actúa como backbone principal de EDA para eventos de dominio entre microservicios y auditoría transaccional, aprovechando su modelo de log distribuido, inmutable y replayable. Azure Service Bus se mantiene en un rol complementario y acotado para comandos que requieren dead-letter queue, reintentos con backoff garantizado y patrones de compensación Saga — donde la semántica de entrega garantizada de cola es más apropiada que el modelo de streaming.
- **Justificación 2:** La elección de Kafka permite una arquitectura Event-Driven pura, donde la trazabilidad y el historial de transacciones financieras son first-class citizens, cumpliendo con los requisitos de alta concurrencia (>5,000 usuarios) y auditoría bancaria sin puntos de fallo adicionales derivados de arquitecturas híbridas. Asimismo, el Confluent Schema Registry garantiza que los contratos de eventos entre Service Domains BIAN evolucionen de forma backward-compatible, aspecto crítico cuando

múltiples consumidores dependen del mismo stream en un entorno bancario regulado.

xi. Cloud Provider: Microsoft Azure

- **Justificación 1:** La integración nativa de los servicios de Azure (AKS, Azure API Management, Azure Key Vault) con las herramientas de CI/CD y monitoreo facilita la adopción de prácticas de DevSecOps y acelera la implementación de los requerimientos de seguridad bancaria.
- **Justificación 2:** La disponibilidad de regiones locales y la madurez de la plataforma en el sector financiero facilitan el cumplimiento de las normativas de soberanía de datos y permiten una configuración de infraestructura multi-region (GRS) necesaria para los planes de contingencia (DRP) del banco.

xii. Auditoría: DB Append-only

- **Justificación 1:** El consumo de eventos mediante Kafka permite que la auditoría ocurra de forma asíncrona, evitando que la latencia del servicio de auditoría bloquee la transacción de negocio del usuario.
- **Justificación 2:** Un modelo append-only es indispensable para el cumplimiento regulatorio de no repudio, asegurando que el historial de transacciones sea inalterable y auditable ante cualquier entidad reguladora.

xiii. Backend for Frontend (BFF)

- **Justificación 1:** Se implementan dos BFFs independientes, uno para la SPA Web y otro para la App Móvil (React Native), permitiendo adaptar la orquestación y el payload de las respuestas a las restricciones técnicas de cada canal. Mientras el BFF Web optimiza la carga de datos estructurados, el BFF Móvil prioriza el formato compacto y la tolerancia a redes inestables, garantizando una latencia  $p95 \leq 8$  segundos y una experiencia de usuario fluida.
- **Justificación 2:** Esta segregación permite que cada BFF actúe como una capa de abstracción dedicada que gestiona sus propios contratos de API. Al mover la lógica de composición (ej. el agregado de datos de perfil, saldos y movimientos) fuera del cliente hacia cada BFF, se reduce drásticamente la complejidad en el frontend, se refuerza la seguridad al ocultar la topología interna de los microservicios y se facilita el mantenimiento independiente de las APIs de cada plataforma frente a cambios en los servicios de dominio.

xiv. Alta Disponibilidad y Resiliencia

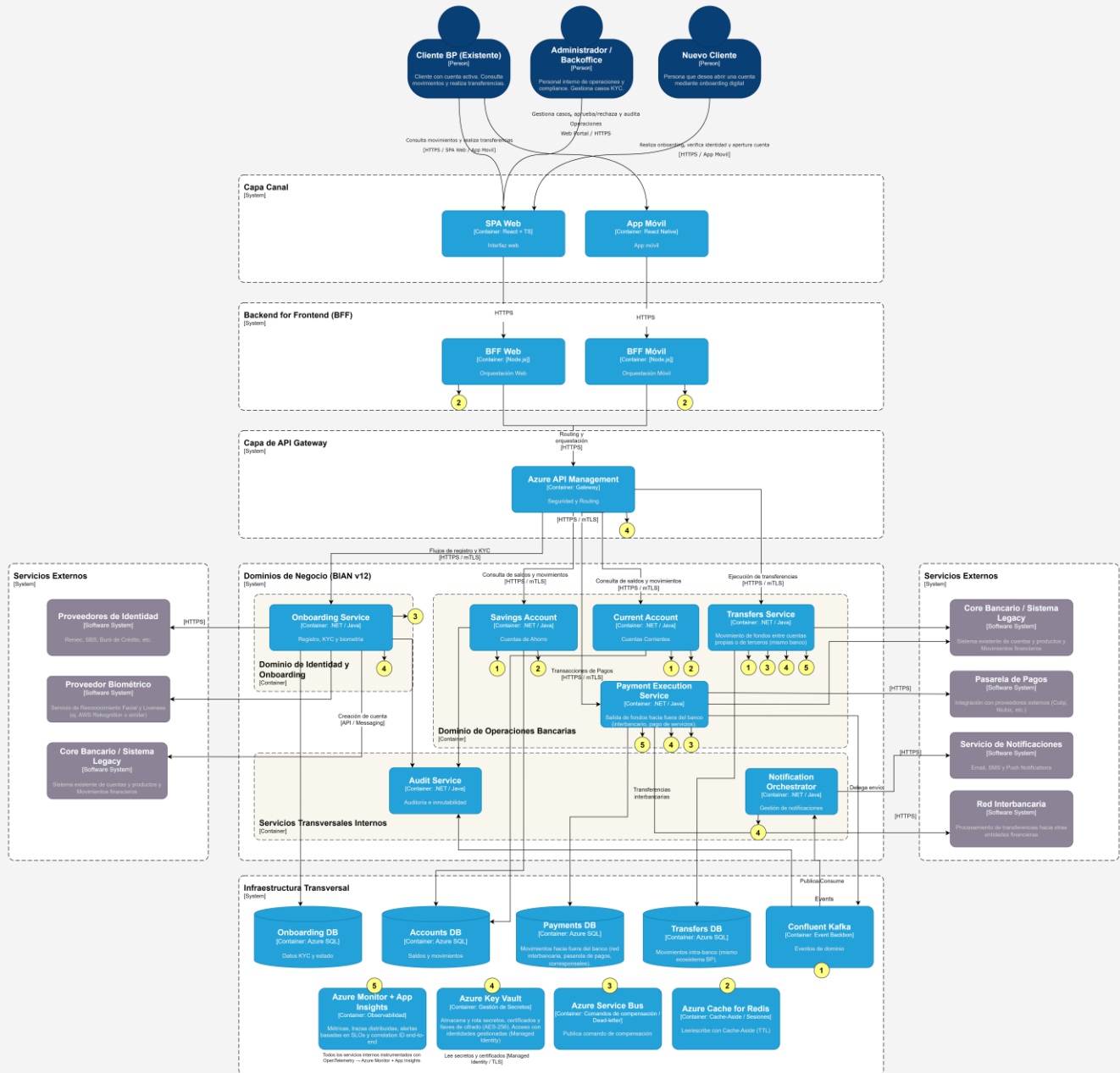
- **Justificación 1:** El despliegue en múltiples Zonas de Disponibilidad (Availability Zones) para AKS y bases de datos asegura que la plataforma sobreviva a la caída de un centro de datos sin intervención manual, garantizando una disponibilidad del 99.95%.
- **Justificación 2:** La implementación de replicación geo-redundante (GRS) define un RPO (Recovery Point Objective) de 1 hora y un RTO (Recovery Time Objective) de 4 horas para servicios críticos, estableciendo expectativas claras de negocio ante un desastre regional catastrófico.

xv. Resiliencia, Excelencia Operativa y Auto-healing

- **Justificación 1 (Resiliencia entre servicios):** Se implementa el patrón Circuit Breaker (Resilience4j) en la comunicación síncrona entre microservicios. Este mecanismo evita el consumo ineficiente de recursos y el efecto "cascada" ante caídas de servicios dependientes, permitiendo una degradación elegante del sistema (fail-fast) y protegiendo la disponibilidad global.
- **Justificación 2 (Operabilidad y Auto-healing):** Se integra el auto-healing mediante health probes y el Auto-scaling de Kubernetes (HPA + Cluster Autoscaler), complementado con la definición de SLOs (Service Level Objectives) enfocados en métricas de negocio. Esto permite que el sistema se recupere automáticamente ante fallos de infraestructura y que el equipo de operaciones detecte anomalías en la experiencia de usuario antes de que se conviertan en incidentes críticos.

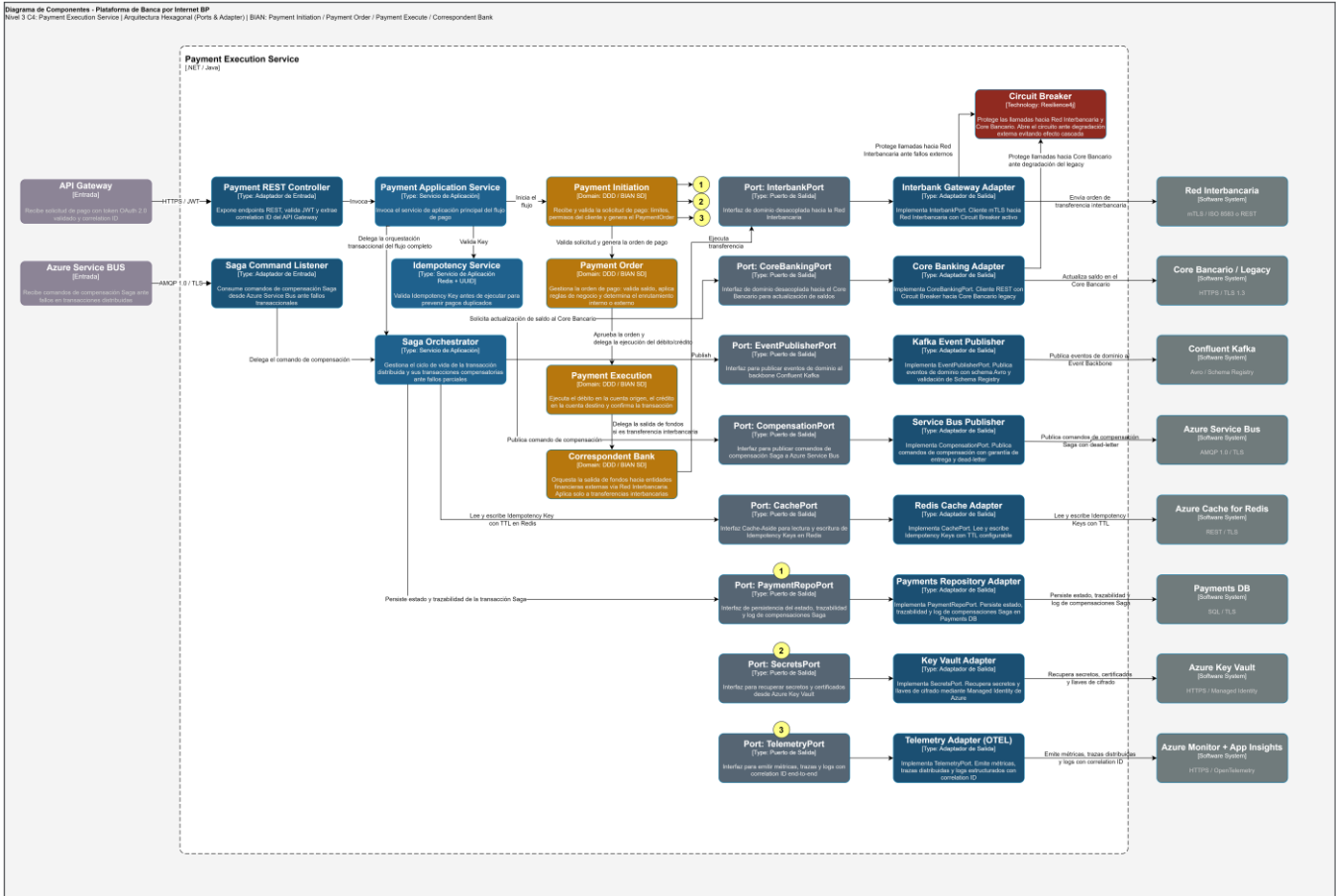
a. Diagrama C4 – Nivel 2 (Contenedores)

Diagrama de Contenedores - Plataforma de Banca por Internet BP  
Nivel 2 C4: Elementos de dominio, infraestructura y sistemas externos



b. Diagrama C4 – Nivel 3 (Componentes)

El diagrama desarrollado se realizó sobre el Payment Execution Service por ser el más rico en patrones (SAGA, idempotencia, Circuit Breaker, hexagonal).



## 6. Resiliencia, Alta Disponibilidad y Seguridad

### a. Alta Disponibilidad y Tolerancia a Fallos

La plataforma garantiza disponibilidad  $\geq 99.95\%$  mediante despliegue multi-zona (AKS + Azure SQL en Availability Zones), escalabilidad horizontal automática (HPA + Cluster Autoscaler), Circuit Breaker (Resilience4j) en todas las llamadas a sistemas externos para evitar efecto cascada, patrón SAGA orquestado para consistencia eventual en flujos distribuidos, e Idempotency Keys para prevenir duplicidad de transacciones ante reintentos.

### b. Recuperación ante Desastres (DR)

Todas las bases de datos críticas se replican geo-redundantemente (GRS) hacia una región secundaria de Azure, con RPO máximo de 15 minutos para servicios de pago y RTO de 1 hora. El failover se activa de forma automatizada mediante Azure SQL Geo-Replication, con simulacros de DR obligatorios cada trimestre.

### c. Auto-healing y Resiliencia

Kubernetes gestiona el auto-healing mediante Liveness y Readiness Probes: pods no responsivos se reinician automáticamente y se excluyen del balanceo de carga hasta su recuperación. Los adaptadores de salida implementan retry con backoff exponencial y jitter para evitar tormentas de reintentos sobre sistemas externos.

### d. Observabilidad

Tres pilares instrumentados con OpenTelemetry: métricas de negocio y de infraestructura en Azure Monitor con alertas basadas en SLOs; trazas distribuidas con correlation ID end-to-end en Application Insights; y logs estructurados (JSON) centralizados con el correlation ID embebido. Las alertas se disparan antes de alcanzar los límites de los RNF (ej. alerta a 6 segundos de latencia p95, antes del límite de 8 segundos).

### e. Seguridad — Zero Trust

Azure API Management como único punto de entrada público con validación JWT, rate limiting y throttling. Comunicación interna entre microservicios protegida con mTLS. Bases de datos y servicios de mensajería expuestos únicamente mediante Private Endpoints. Secretos y certificados gestionados en Azure Key Vault con acceso por Managed Identity. Cifrado AES-256 en reposo y TLS 1.3 en tránsito. Step-up Authentication (MFA o reconocimiento facial) para operaciones de alto riesgo. Cumplimiento de OWASP Top 10, PCI-DSS, Ley 29733 y regulaciones SBS.

## 7. Estrategia de Migración y Evolución

La transición desde el sistema legacy hacia la nueva plataforma se realiza mediante el patrón **Strangler Fig**, garantizando zero downtime en todo momento. El **Azure API Management** actúa como fachada unificada que enruta el tráfico hacia la nueva plataforma o hacia el sistema legacy según el estado de migración de cada capacidad, retirando progresivamente el sistema existente por fases hasta su descomisión completa.

La migración se estructura en cuatro fases:

- **Fase 1 — Fundación (Meses 1–3):** despliegue de infraestructura base en Azure (AKS, API Management, Confluent Kafka, Key Vault, Azure Monitor), implementación del flujo de autenticación OAuth 2.0 + PKCE y del Onboarding Service con reconocimiento facial.
- **Fase 2 — Gestión de Cuentas (Meses 4–6):** migración de Savings Account Service y Current Account Service con enrutamiento gradual de tráfico y Cache-Aside con Redis como protección al Core Bancario legacy durante la transición.
- **Fase 3 — Motor de Pagos y Transferencias (Meses 7–9):** migración de la cadena de pagos (Payment Initiation, Payment Order, Payment Execution, Correspondent Bank) mediante canary deployment progresivo (5% → 25% → 50% → 100%) con métricas de negocio monitoreadas en cada incremento.
- **Fase 4 — Cierre y Descomisión (Meses 10–12):** migración de Audit Service y Notification Orchestrator, validación final de consistencia de datos históricos y descomisión del sistema legacy.

Durante todo el proceso se aplican **feature flags** para activar o desactivar capacidades en caliente sin redespliegue, y **backward compatibility en Confluent Schema Registry** para garantizar la coexistencia de consumidores legacy y nuevos durante la transición.

## 8. Conclusiones y Recomendaciones

### a. Conclusiones

La arquitectura propuesta para la plataforma de banca por internet de BP se sustenta en ocho pilares que en conjunto garantizan una solución moderna, segura y preparada para el crecimiento:

- **Alineamiento BIAN v12:** la descomposición en microservicios basada en Service Domains (Customer Onboarding, KYC, Savings Account, Current Account, Payment Initiation, Payment Order, Payment Execution, Correspondent Bank, Customer Notification y Financial Transaction History) elimina la arbitrariedad en los límites de servicio y establece un lenguaje común con el regulador y la industria bancaria internacional.
- **Domain-Driven Design (DDD):** cada microservicio encapsula un Bounded Context con reglas de negocio propias e independientes, garantizando que los cambios en un dominio no impacten a otros y facilitando la evolución autónoma de cada capacidad.
- **Arquitectura Hexagonal (Ports & Adapters):** el aislamiento de la lógica de dominio respecto a los adaptadores de infraestructura permite cambiar proveedores externos (biométrico, notificaciones, red interbancaria) sin afectar el núcleo de negocio, reduciendo el riesgo tecnológico a largo plazo.
- **Escalabilidad y Performance:** la combinación de AKS con HPA y Cluster Autoscaler, Cache-Aside con Azure Cache for Redis y Confluent Kafka como backbone de eventos garantiza el soporte de más de 5,000 usuarios concurrentes con tiempos de respuesta  $\leq 8$  segundos en p95, absorbiendo picos de demanda sin degradación del servicio.
- **Resiliencia Operativa:** los patrones de Circuit Breaker (Resilience4j), SAGA orquestado, Idempotency Keys y Auto-healing de Kubernetes garantizan que los fallos parciales de sistemas externos no se propaguen al usuario final, sosteniendo el SLA de 99.95% de disponibilidad.
- **Seguridad por diseño:** el modelo Zero Trust, mTLS entre servicios internos, Azure Key Vault para gestión de secretos y Step-up Authentication para operaciones de alto riesgo conforman una arquitectura de seguridad en capas que protege los datos financieros del cliente en cada punto de la plataforma.
- **Cumplimiento normativo:** la auditoría inmutable sobre Confluent Kafka, el cifrado AES-256 en reposo y TLS 1.3 en tránsito, y la alineación a OWASP Top 10, PCI-DSS, Ley 29733 y regulaciones SBS garantizan el cumplimiento regulatorio sin comprometer la experiencia del usuario.
- **Migración con Strangler Fig:** la transición progresiva desde el sistema legacy mediante el patrón Strangler Fig, con enrutamiento dual en Azure API Management, canary deployments y feature flags, garantiza zero downtime durante todo el ciclo de migración de 12 meses.

### b. Recomendaciones

- **Sobre BIAN y DDD:** formalizar el mapeo de Service Domains como Architecture Decision Records (ADRs) vivos, actualizados ante cambios de contexto o incorporación de nuevos productos financieros. Presentar el alineamiento BIAN ante la SBS como evidencia de adopción de estándares internacionales, lo que puede acelerar procesos de certificación y auditoría regulatoria.



- **Sobre Arquitectura Hexagonal:** establecer desde la Fase 1 plantillas base (scaffolding) de microservicio hexagonal estandarizadas para todos los equipos, evitando implementaciones inconsistentes del patrón que generen deuda técnica entre dominios.
- **Sobre Escalabilidad y Performance:** definir y validar los umbrales de HPA y los TTLs de Redis con pruebas de carga reales antes del go-live de cada fase, usando los RNF del capítulo 2 como criterios de aceptación no negociables.
- **Sobre Resiliencia Operativa:** invertir en observabilidad desde el día 1 — instrumentar OpenTelemetry y Azure Monitor antes del primer despliegue en producción. Sin trazabilidad distribuida desde el inicio, el diagnóstico de incidentes durante la migración será exponencialmente más costoso.
- **Sobre Seguridad y Cumplimiento:** ejecutar un Security Assessment independiente al finalizar cada fase de migración, validando que los controles Zero Trust, mTLS y Key Vault estén correctamente implementados antes de aumentar el porcentaje de tráfico sobre la nueva plataforma.
- **Sobre Strangler Fig:** no avanzar a la siguiente fase de migración sin validar la consistencia de datos entre el sistema legacy y la nueva plataforma. Mantener los feature flags activos como mecanismo de rollback inmediato durante al menos 30 días después de cada migración de capacidad crítica.

## 9. Links de Referencia

Repositorio GIT: [https://github.com/jairtorresarteaga/test\\_devsu](https://github.com/jairtorresarteaga/test_devsu)